

Projeto e Implementação: Projeto Detalhado

Prof. Lucinéia Heloisa Thom

INF01127 - Engenharia de Software N

Projeto e Implementação



PADRÕES DE PROJETO

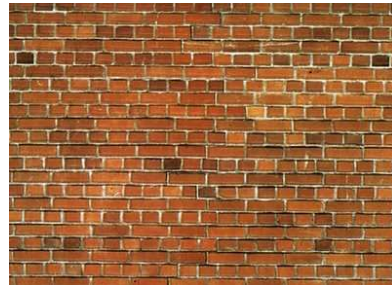
Padrão de Projeto

Um padrão descreve um problema no nosso ambiente e o cerne da sua solução, de tal forma que você possa usar essa solução mais de um milhão de vezes, sem nunca fazê-lo da mesma maneira

(Christopher Alexander)



Interface

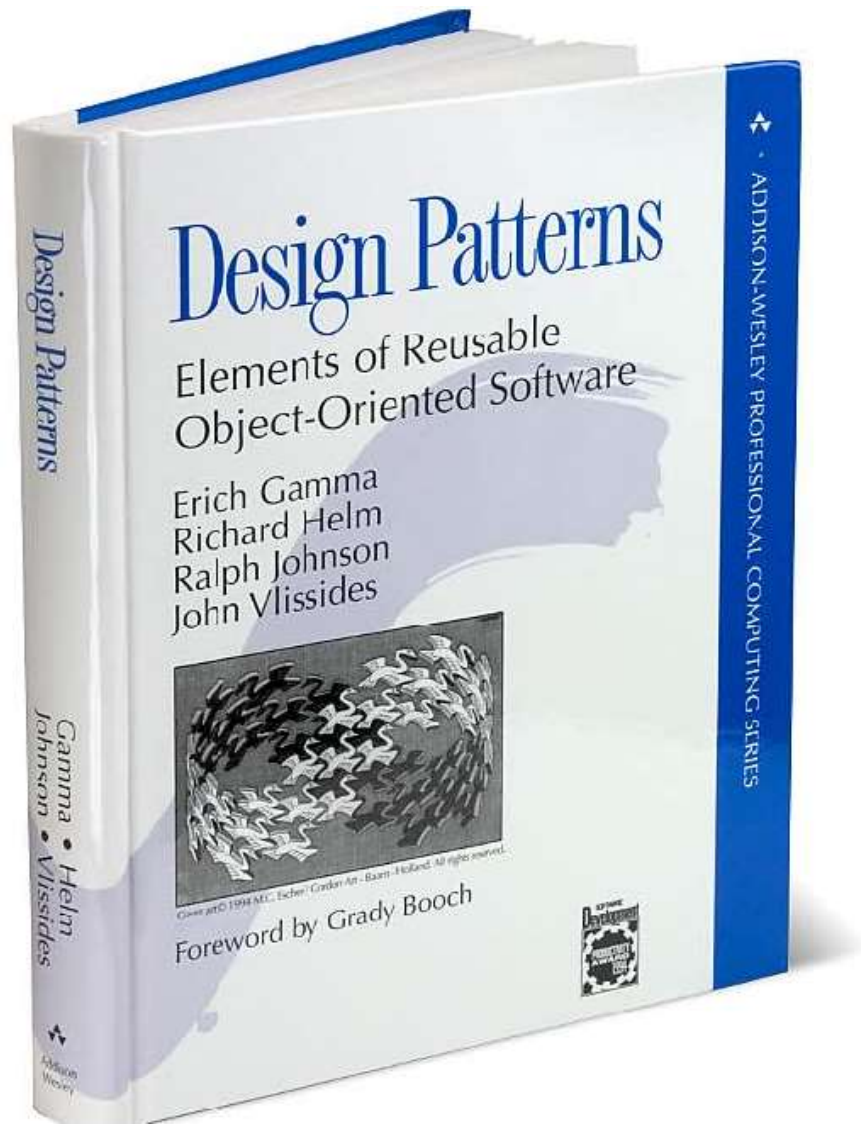


Objeto

Elementos de um Padrão

- **Nome**
 - Referência para descrever um problema de projeto, suas soluções e consequências em uma ou duas palavras
- **Problema**
 - Descreve em que situação usar o padrão. Explica o problema e o contexto. Pode incluir uma lista de condições que devem ser satisfeitas para que o padrão seja aplicado
- **Solução**
 - Descreve os elementos que descrevem o padrão de projeto, seus relacionamentos, suas responsabilidades e colaborações
- **Consequências**
 - São os resultados e análises das vantagens e desvantagens da aplicação do padrão

Padrões de Projeto



- Catálogo de 23 padrões
- Não apresenta padrão para um domínio de aplicação específico
- Padrões do GoF (*gangue dos quatro*) representam o estado-da-prática em boas construções de projeto orientado a objetos
- É comum encontrar no detalhamento de padrões específicos de domínio a ocorrência de algum dos padrões do GoF

Padrões de Projeto

		Purpose		
		Creational	Structural	Behavioral
Scope	Class	Factory Method	Adapter (class)	Interpreter Template Method
	Object	Abstract Factory Builder Prototype Singleton	Adapter (object) Bridge Composite Decorator Flyweight Facade Proxy	Chain of Responsibility Command Iterator Mediator Memento Observer State Strategy Visitor

Scope: domain over which a pattern applies

Purpose: reflects what a pattern does

- **Propósito**

- **Criação**

- Criação dos objetos

- **Estruturais**

- Composição de classes ou de objetos

- **Comportamentais**

- Interação entre classes e distribuição de responsabilidades

- **Escopo**

- **Classe (estáticos)**

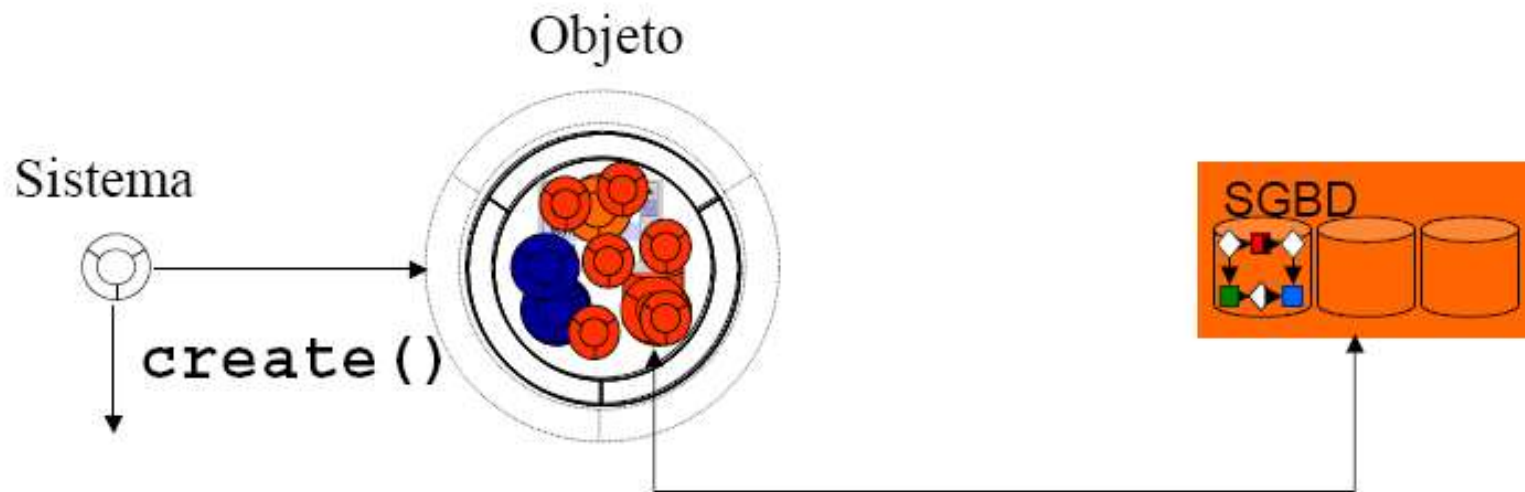
- Relacionamentos entre classes e subclasses

- **Objeto (dinâmico)**

- Relacionamentos entre objetos (podem mudar em tempo de execução)

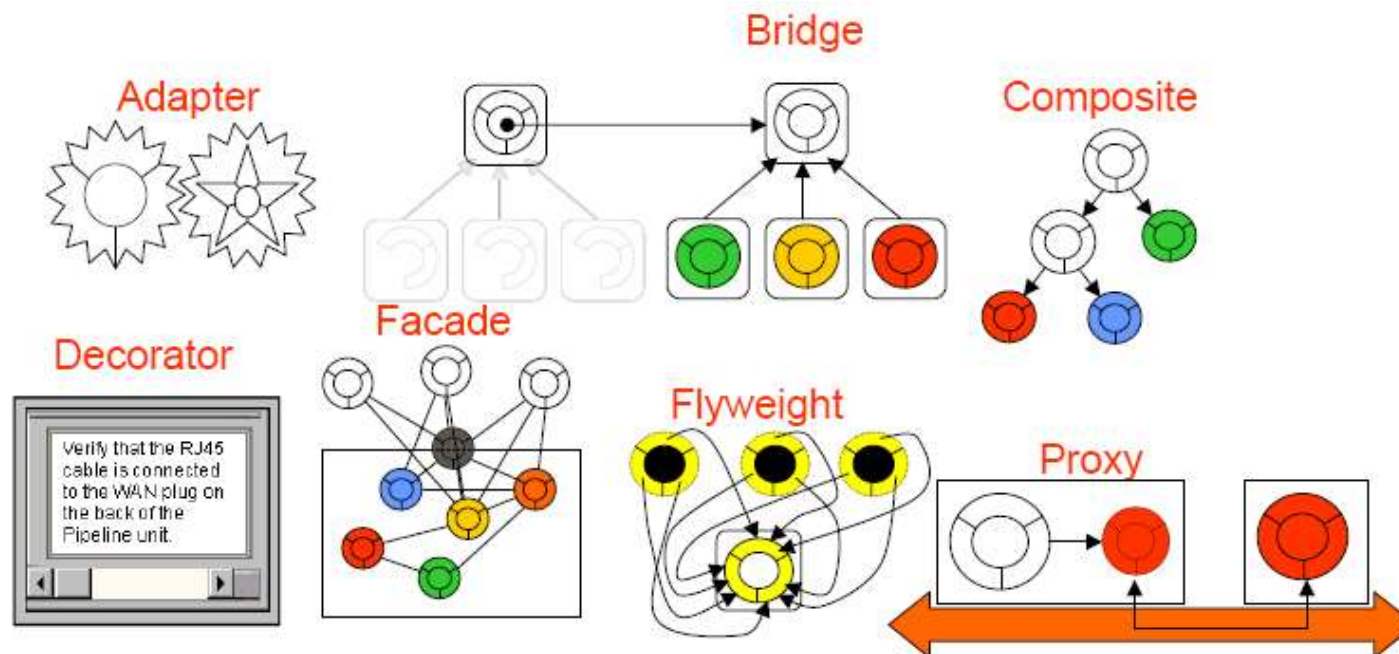
Padrões de Projeto

- Classificação
 - Padrões de Criação
 - Abstraem o processo de instanciação
 - Tornam um sistema independente da forma como os objetos são criados, compostos e representados



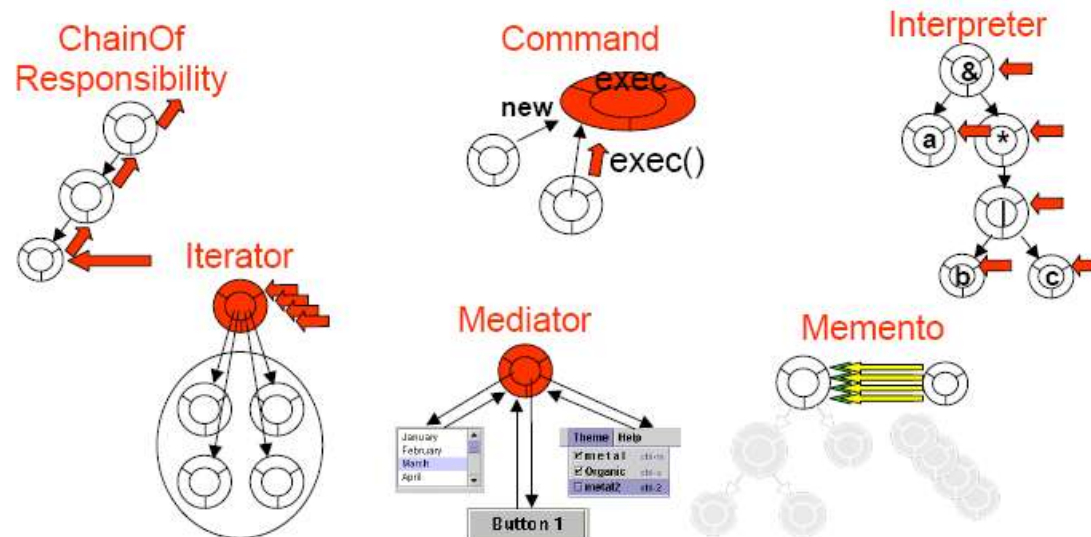
Padrões de Projeto

- Classificação
 - Padrões Estruturais
 - Lidam com a composição de classes (ou objetos) para formar grandes estruturas no sistema



Padrões de Projeto

- Classificação
 - Padrões Comportamentais
 - Caracterizam a forma como classes (ou objetos) interagem (comunicações)
 - Distribuem responsabilidade



Padrões de Projeto: Exemplos

- Singleton
 - Assegurar que uma classe tenha uma única instância e prover um ponto de acesso global a esta instância (ex.: impressora com somente um *spooler*)
- Fachada
 - Prover uma interface simplificada para um sistema complexo (reduz acoplamento, simplificação) – ex.: uma interface simples para a maioria dos clientes
- Abstract Factory
 - Quando um sistema deve ser independente de como seus produtos são criados, compostos e representados
 - Quando uma família de produtos relacionados foi projetada para uso conjunto e você deve implementar essa restrição
 - Quando você quer fornecer uma biblioteca de classes e quer revelar sua interface e não sua implementação

Padrões de Projeto: Exemplos

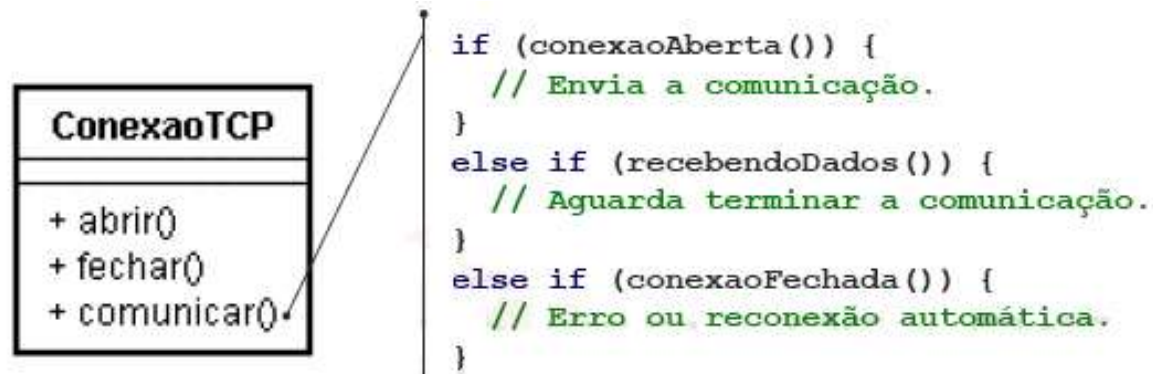
- DAO (Objeto de Acesso a Dados)
 - Separar o modelo de dados da aplicação da tecnologia de persistência de dados
 - Promover isolamento e flexibilidade, principalmente, quando se deseja, por exemplo, mudar o SGBD
- Ao considerar a persistência, cada coleção funciona como um DAO
 - DAOProduto (incluir, remover, pesquisar) → Catalogo
 - DAOVenda (incluir, remover, pesquisar)
- Pode ter mais ou menos inteligência
 - Persistência pura
 - Montagem de objetos na memória (ex: venda – item – produto)

Padrões de Projeto: State

- Classificação
 - Comportamental
- Também conhecido como
 - *Objects for States*
- Propósito
 - Permitir a um objeto alterar o seu comportamento quando o seu estado interno mudar

Padrões de Projeto: State

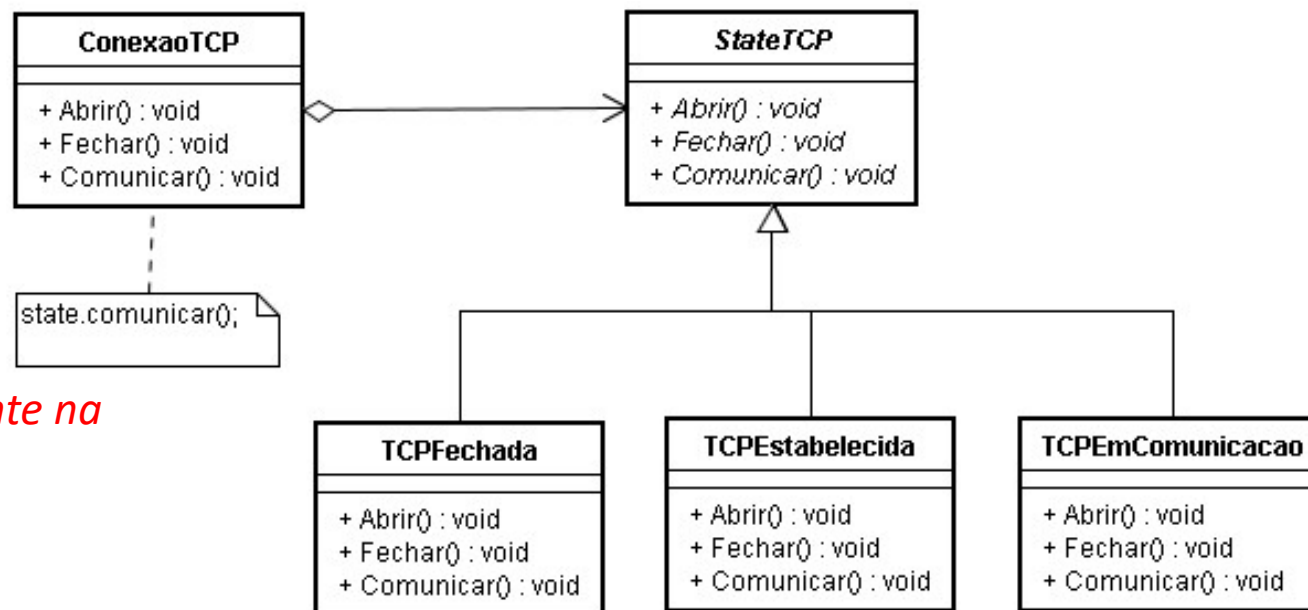
- Motivação
 - Exemplo de Problema



- Um objeto responde de maneira distinta dependendo do seu estado

Padrões de Projeto: State

- Exemplo de Solução
 - Conexão possui um objeto que representa o seu estado e que executa as tarefas dependentes de estado



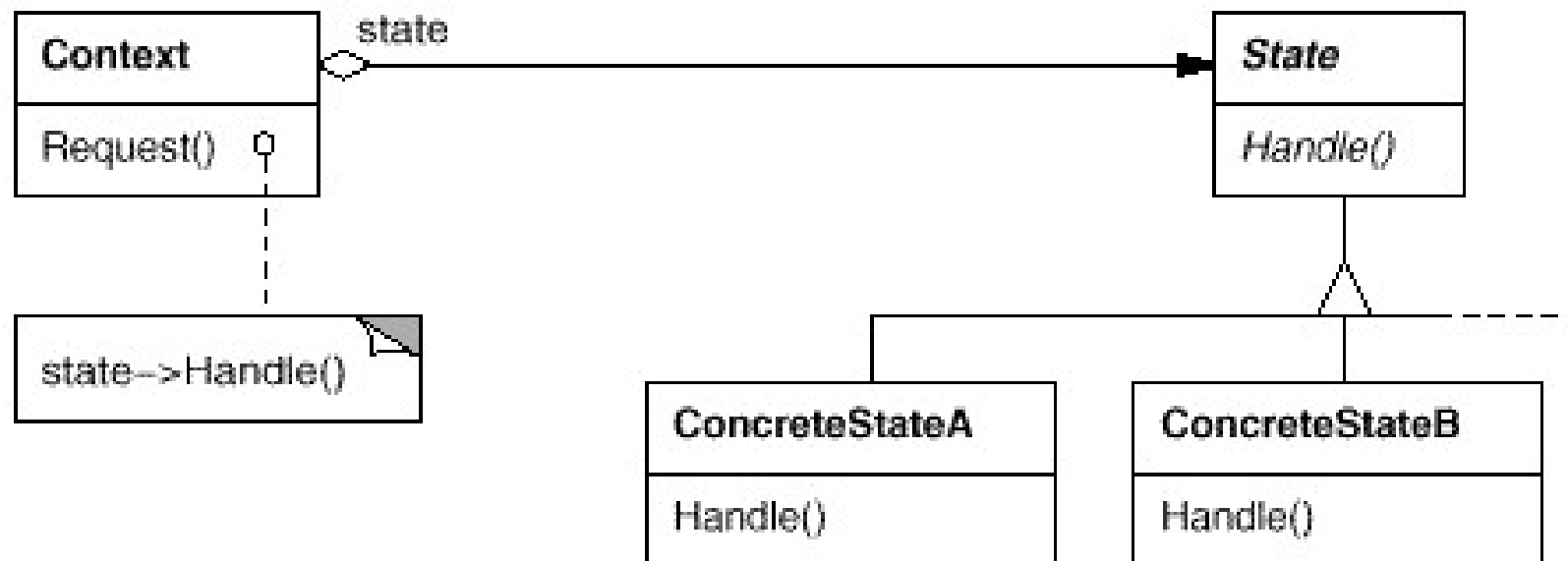
*Estado corrente na
conexão TCP*

Padrões de Projeto: State

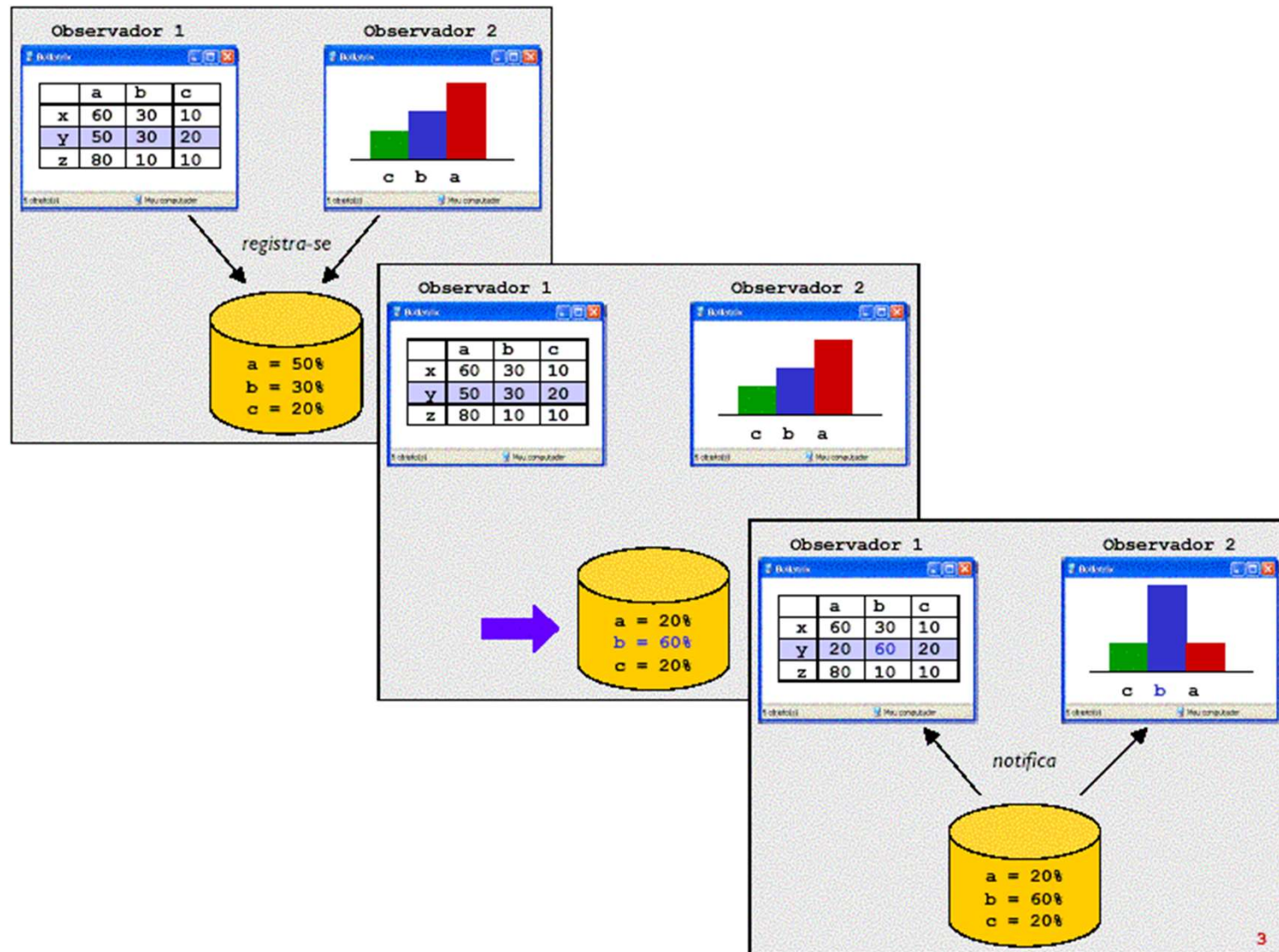
- Aplicabilidade
 - Comportamento de um objeto depende do seu estado que é alterado em tempo de execução
 - Operações de um objeto possuem condicionais grandes, de várias alternativas (sintoma do caso anterior)
 - Frequentemente, muitas operações irão conter essa mesma estrutura condicional
 - Estados são usualmente representados por uma ou mais constantes
 - State coloca cada ramo de comando condicional em uma classe separada, tratando os estados dos objetos como objetos propriamente ditos

Padrões de Projeto: State

- Estrutura
 - Questão: quem define a transição de estados?



Padrões de Projeto: Observer

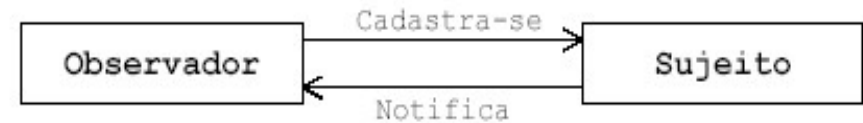


Padrões de Projeto: Observer

- Propósito

- Dependência de um-para-muitos entre objetos

- Quando um objeto muda de estado, todos seus dependentes são notificados e atualizados automaticamente



- Aplicabilidade

- Abstração tem dois aspectos, um dependente do outro

- Encapsular estes aspectos em objetos separados permite variação e reuso independentemente

- Mudança em um objeto requer alterar outros

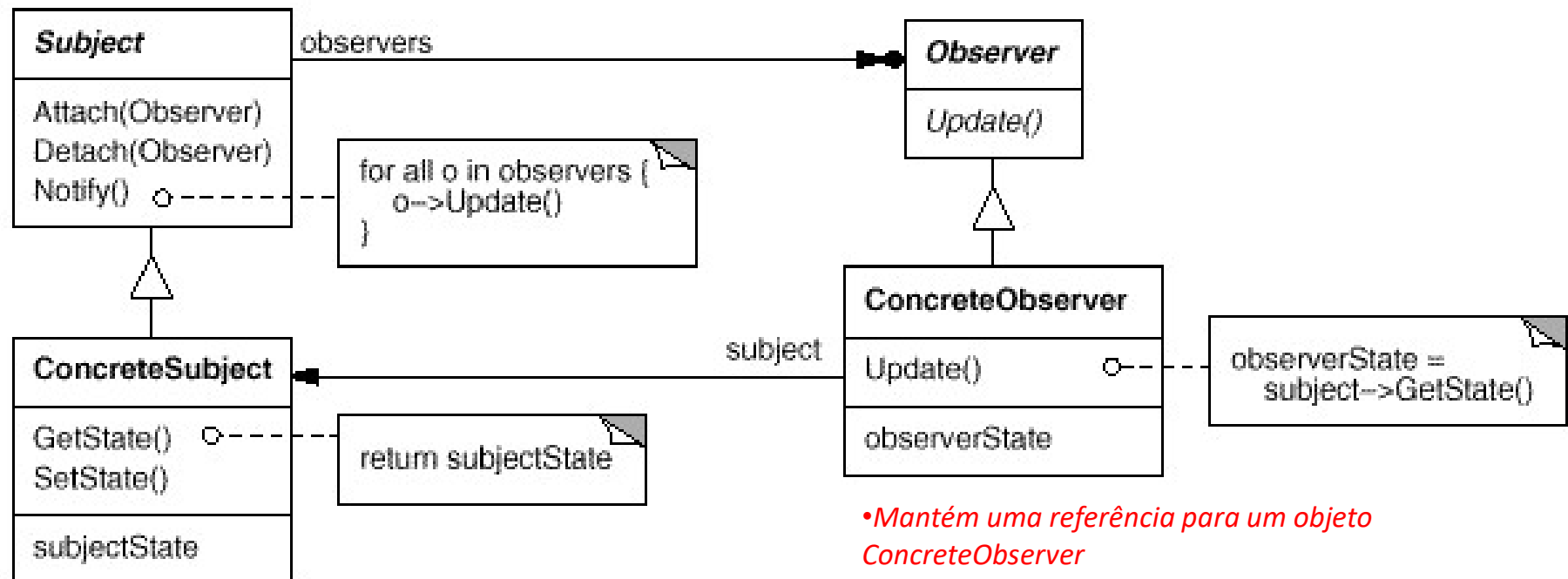
- Não se sabe quantos objetos precisam ser alterados

- Objeto capaz de notificar outros objetos sem presumir quem são esses objetos

Padrões de Projeto: Observer

- Estrutura

Conhece seus observadores



Interface de atualização para objetos que deveriam ser notificados sobre mudanças em um Subject

- Armazena estados de interesse para objetos **ConcreteObserver**
- Envia uma notificação para os seus observadores quando seu estado muda

- Mantém uma referência para um objeto **ConcreteObserver**
- Armazena estados que deveriam permanecer consistentes com os do **Subject**
- Implementa a interface de atualização de **Observer**, para manter seu estado consistente com o do **subject**

Padrões de Projeto

- Não devem ser utilizados indiscriminadamente
 - Aumenta complexidade
 - Baixa desempenho
 - Preço pago pela flexibilidade obtida deve valer a pena

Projeto e Implementação



IMPLEMENTAÇÃO

Implementação

- Foco
 - Discutir questões que não são abordadas em material sobre programação



Questões de Implementação

- **Reuso**
 - Software moderno é construído através da reutilização de componentes ou sistemas existentes
 - Ao desenvolver software
 - Fazer o máximo uso possível de código existente
- **Gerenciamento de Configuração**
 - Durante o processo de desenvolvimento
 - Manter o controle das muitas versões diferentes de cada componente de software em um sistema de gerenciamento de configuração
- **Desenvolvimento Host-target**
 - Software produzido não costuma executar no mesmo computador como no do ambiente de desenvolvimento de software
 - Desenvolver em um computador (o sistema host) e executá-lo em um computador separado (o sistema target)

Reuso

- Da década de 1960 à de 1990
 - Maior parte de software novo era desenvolvido a partir do zero, escrevendo todo o código em uma linguagem de programação de alto nível
 - Único reuso de software significativo foi **a reutilização de funções e objetos em bibliotecas de linguagem de programação**
- Pressão de custos e cronograma fazem com que essa abordagem se tornasse cada vez mais inviável
 - Especialmente para sistemas comerciais e baseado na Internet
- Abordagem de desenvolvimento baseada em torno do reuso de software existente surgiu
 - Agora é largamente usado para software empresarial e científico
 - Leitura: <http://www.cse.wustl.edu/~schmidt/reuse-lessons.html>

Níveis de Reuso

- Nível de **Abstração**
 - Não se reusa software diretamente, mas o conhecimento de abstrações de sucesso no projeto de software
- Nível do **Objeto**
 - Reuso de objetos de uma biblioteca ao invés de escrever o código
- Nível de **Componente**
 - Componentes são coleções de objetos e classes de objetos que são reutilizados em sistemas de aplicação
- Nível de **Sistema**
 - Reuso de sistemas de aplicação inteiros

Custos do Reuso

- Custos do tempo gasto na **procura** de software para reuso
 - Avaliar se ele atende às necessidades
- Quando aplicável, os custos de **aquisição** do software reutilizáveis
 - No caso de grandes sistemas, estes custos podem ser muito elevados
- Custos de **adaptação** e configurar os componentes de software reutilizáveis ou sistemas para refletir os requisitos do sistema que está sendo desenvolvido
- Custos de **integração** de elementos de software reutilizáveis entre si (se software de diferentes fontes estiver sendo usando) e com o novo código sendo desenvolvido

Gerenciamento de Configuração

- Nome dado ao processo geral de gerenciamento de um sistema de software evolutivo
- Objetivo
 - Apoiar o processo de integração do sistema de modo que todos os desenvolvedores possam acessar o código do projeto e os documentos de forma controlada
 - Descobrir que alterações foram feitas
 - Compilar e ligar componentes para criar um sistema
- Responder as seguintes perguntas:
 - O que mudou e quando?
 - Por que mudou?
 - Quem fez a mudança?
 - Podemos reproduzir esta mudança?

Gerenciamento de Configuração

- Atividades
 - **Gerenciamento de versão**
 - Acompanhar as diferentes versões de componentes de software
 - Sistemas de gerenciamento de versão incluem instalações para coordenar o desenvolvimento de vários programadores
 - **Integração de Sistema**
 - Ajudar os desenvolvedores a definir quais versões de componentes são usados para criar cada versão de um sistema
 - Descrição utilizada para construir um sistema automaticamente através da compilação e ligação dos componentes necessários
 - **Rastreamento de problemas**
 - Permitir aos usuários reportar bugs e outros problemas, e permitir que todos os desenvolvedores vejam quem está trabalhando sobre estes problemas e quando eles são corrigidos

Desenvolvimento Host-target

- Maioria dos software é desenvolvido em um computador (**host**)
 - Mas é executado em uma máquina separada (**target**)
- De modo mais geral, podemos falar de uma plataforma de desenvolvimento e uma plataforma de execução
 - **Plataforma é mais do que apenas o hardware**
 - Inclui o sistema operacional instalado além de outros software de apoio
 - Como um sistema de gerenciamento de banco de dados ou
 - Para plataformas de desenvolvimento, um ambiente de desenvolvimento interativo
- Plataforma de desenvolvimento normalmente tem **diferentes softwares instalados** além da plataforma de execução
 - Essas plataformas podem ter diferentes arquiteturas

Plataformas de Desenvolvimento

- **Compilador** integrado e sistema de **edição** com *syntax highlighting* que permite criar, editar e compilar o código
- Sistema de **depuração**
- Ferramentas de **edição gráfica**
 - Como ferramentas para editar modelos UML
- Ferramentas de **teste**
 - Tais como JUnit que podem automaticamente executar uma série de testes em uma nova versão de um programa
- Ferramentas de **apoio** a **projetos**
 - Ajudam a organizar o código para diferentes projetos de desenvolvimento

Integrated Development Environments (IDEs)

- Ferramentas de desenvolvimento de software muitas vezes são agrupadas para criar um ambiente de desenvolvimento integrado (IDE)
 - E.g. Eclipse, VisualStudio
- Conjunto de ferramentas de software que suporta diferentes aspectos do desenvolvimento de software
 - Dentro de algum framework em comum e interface de usuário
- Criado para apoiar o desenvolvimento de uma linguagem de programação específica
 - E.g. Java
- IDE da linguagem podem ser desenvolvido especificamente, ou pode ser uma instanciação de um IDE de propósito geral, com ferramentas de suporte à linguagem específica

Desenvolvimento Open Source

- Abordagem para desenvolvimento de software em que o código-fonte de um sistema de software é publicado e voluntários são convidados a participar no processo de desenvolvimento
- Raízes na *Free Software Foundation* (www.fsf.org)
 - Defende que o código fonte não deve ser proprietário
 - Deve estar sempre disponível para os usuários analisar e modificar o que quiserem
- Software open source estendeu essa idéia usando a Internet para recrutar uma população muito maior de desenvolvedores voluntários
 - Muitos deles são também usuários do código

Sistemas Open Source

- Produto mais conhecido de fonte aberta é, evidentemente, o sistema operacional **Linux**
 - Largamente utilizado como um sistema de servidor e, cada vez mais, como um ambiente de trabalho
- Outros produtos importantes
 - Linguagem de Programação
 - **Java**
 - Servidor web
 - **Apache**
 - Sistema de gerenciamento de banco de dados
 - **mySQL**

Negócio Open Source

- Cada vez mais empresas estão usando uma abordagem open source para o desenvolvimento
- Seu **modelo de negócio** não é dependente da venda de um produto de software
 - Mas na venda de **suporte** para esse produto
- Acredita-se que o envolvimento da comunidade de código aberto permitirá que o software a ser desenvolvido de forma mais barata, mais rápida e vai criar uma comunidade de usuários para o software

Licença Open Source

- Princípio fundamental de desenvolvimento open-source
 - Código fonte deve ser livremente disponível
 - Isso não significa que qualquer um pode fazer o que desejar com esse código.
- Legalmente, o desenvolvedor do código (seja uma empresa ou um indivíduo) ainda possui o código
 - Pode colocar restrições sobre a forma como ela é usada, incluindo condições juridicamente vinculativos em uma licença de software open source
- Alguns desenvolvedores open source acreditam que
 - Se um componente de código aberto é usado para desenvolver um novo sistema
 - Sistema também deve ser de código aberto.
- Outros estão dispostos para permitir o seu código para ser utilizado sem esta restrição
 - Sistemas desenvolvidos podem ser proprietários e vendidos como sistemas de código fechado

Modelos de Licença

- **Licença GNU General Public (GPL)**
 - Chamada licença "recíproca"
 - Se você usar o software open source, que está licenciado sob a licença GPL, então você deve fazer que o software seja open source
- **Licença GNU Lesser General Public (LGPL)**
 - Variante da licença GPL
 - Você pode escrever componentes que apontam para código open source sem ter que publicar os fontes desses componentes
- **Licença Berkley distribuição padrão (BSD)**
 - Licença não-recíproca
 - Você não é obrigado a publicar as alterações ou modificações feitas no código open source
 - Você pode incluir o código em sistemas proprietários, que são vendidos

Gerenciamento de Licença

- Estabelecer um sistema para manter as informações sobre os componentes open source que são baixados e usados
- Estar ciente dos diferentes tipos de licenças e compreender como um componente está licenciado antes de ser usado
- Esteja ciente dos caminhos de evolução dos componentes
- Educar as pessoas sobre open source
- Ter sistemas de auditoria em vigor
- Participar da comunidade open source

Referências

- Leitura recomendada
 - Craig Larman. Utilizando UML e Padrões. Bookman. (2d edição mais enxuta, 3ed mais completa)
 - Diagramas de Sequência : cap 10
 - Contratos: cap 11
 - GRASP: 17
- Leitura sugerida
 - Gamma, E. et al. Padrões de Projeto: Soluções Reutilizáveis de Software Orientado a Objetos. Bookman.

Perguntas?

- Este material tem contribuições de
 - Ingrid Nunes
 - Karin Becker
 - Lucinéia Thom
 - Marcelo Pimenta